MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Done by: st. gr. FAF-242, Temciuc Adelina

Report

Practical Work No. 1 and No. 2

Operating Systems

Topic: Linux Fundamentals and Exploring the xv6 Operating System

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

1 AIM OF THE WORK

1.1 Purpose of the practical works

The purpose of the two practical works is to study how an operating system manages its main resources and how user programs interact with the kernel. The first work is performed on a Linux system running under WSL and examines files, processes, memory and devices through standard command-line tools. The second work is performed on xv6, a small Unix-like teaching operating system, and examines its shell, its system calls and its source code structure.

The practical works aim to:

- familiarize students with Linux file ownership, permissions and the directory hierarchy;
- inspect running processes and their states through `ps`, `top` and the `/proc` filesystem;
- analyze RAM and swap usage of the system and of an individual process;
- identify block and character devices and the relation between devices and filesystems;
- explore the xv6 shell and the user programs it provides;
- study the system call interface between user space and the kernel in xv6;
- implement a new user program in xv6 that relies on an existing system call.

Overall, the laboratory works connect theoretical knowledge about operating systems with direct practical observation of a real and of an educational kernel.

2 THEORETICAL BACKGROUND

2.1 Files, ownership and permissions in Linux

In Linux every file belongs to exactly one user (the owner) and one group. Access rights are defined separately for three classes of users: the owner, the group and all other users. For each class three rights may be granted: read (r), write (w) and execute (x). The rights are commonly written in octal form, where read has the value 4, write has the value 2 and execute has the value 1; for example, the mode `600` grants read and write access to the owner only [1].

The directory tree starts at the root directory `/`. Standard top-level directories have fixed purposes, for example `/home` stores the personal directories of regular users and `/etc` stores system-wide configuration files [2].

2.2 Processes

A process is a program in execution, together with its memory, open files and execution state. Each process is identified by a unique process identifier (PID). The first process started by the kernel has PID 1 and is responsible for initializing the rest of the user space. The kernel exposes information about every process through the virtual `/proc` filesystem, where the file `/proc/<PID>/status` contains, among others, the state of the process and its memory usage [3].

2.3 Memory and swap

Physical memory (RAM) is shared between the kernel, running processes and the page cache. Swap is an area on secondary storage that the operating system uses as an extension of RAM: when memory becomes insufficient, inactive pages are moved to swap. The resident set size (VmRSS) of a process shows how much of its memory is currently held in RAM [3].

2.4 Devices and the "everything is a file" principle

Linux represents hardware devices as special files located in the `/dev` directory. Block devices (such as disks) transfer data in fixed-size blocks, while character devices (such as terminals) transfer data as a stream of characters. Because devices, pipes and many kernel interfaces are accessed through ordinary file operations such as `open`, `read` and `write`, the Unix design is often summarized as "everything is a file" [4].

2.5 The xv6 operating system and system calls

xv6 is a re-implementation of Unix Version 6 for the RISC-V architecture, developed at MIT for teaching purposes [4]. Its source code is divided into the `kernel/` directory, which contains the operating

system itself, and the `user/` directory, which contains user programs. User programs cannot access hardware directly; instead, they request services from the kernel through system calls such as `fork`, `exec`, `read`, `write` and `sleep`, which are declared in `user/user.h` and implemented in the kernel [5].

3 LINUX FUNDAMENTALS: FILES, PROCESSES, MEMORY AND DEVICES

3.1 File ownership, permissions and directory structure

3.1.1 File ownership

To determine the owner and the group associated with the file `note.txt`, the following command was executed:

```
ls -l note.txt
```

The output was:

```
-rw------- 1 adi adi 0 Sep 28 16:30 note.txt
```

From this output, the owner of the file is **adi** and the group of the file is also **adi**. The first occurrence of `adi` represents the user who owns the file, while the second occurrence represents the group associated with the file.

3.1.2 File permissions after chmod 600

The file was created and its permissions were changed using the following commands:

```
touch note.txt
chmod 600 note.txt
ls -l note.txt
```

The resulting permission string was `-rw-------`. The meaning of its ten characters is presented in table 3.1.

Table 3.1 – Meaning of the permission string -rw-------

Characters	Meaning
`-`	The object is a regular file
`rw-`	The owner has read and write permissions
`---`	The group has no permissions
`---`	Other users have no permissions

Therefore, the permission value `600` means that the owner can read and modify the file, while members of the group and all other users have no access to it. The creation of the file and the configuration of its permissions are shown in figure 3.1, and the resulting ownership and permissions are shown in figure 3.2.

```
adi@Pikachu:/etc$ cd ~/os-lab1
adi@Pikachu:~/os-lab1$ mkdir demo && cd demo
adi@Pikachu:~/os-lab1/demo$ echo "hello operating systems" > note.txt
adi@Pikachu:~/os-lab1/demo$
```

Figure 3.1 – File creation and permission configuration

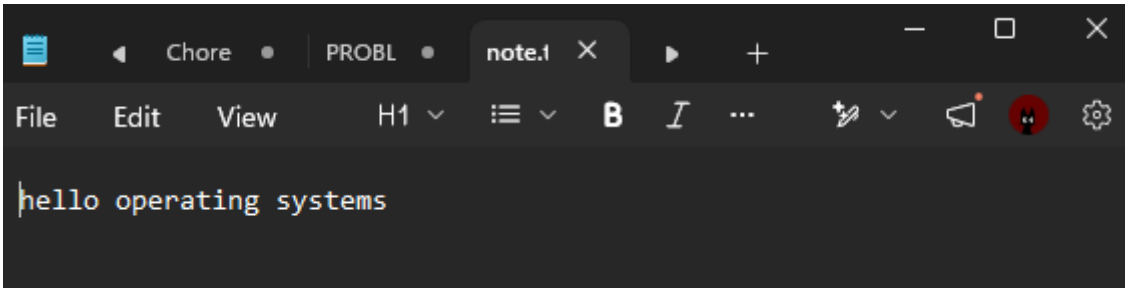


Figure 3.2 – File ownership and permissions displayed in the terminal

3.1.3 Directories under the root directory

The root directory `/` was inspected using the following commands:

```
cd /
ls
```

Two directories located directly under `/` are:

- `/home` contains the home directories of regular users, for example `/home/adi` holds the personal files of the user `adi`;
- `/etc` contains the system-wide configuration files used by the operating system and by installed services.

The output `-rw------- 1 adi adi 0 Sep 28 16:30 note.txt` clearly demonstrates how Linux represents file ownership and access permissions in a single line.

3.2 Process management and /proc inspection

3.2.1 PID of process number 1

The running processes were inspected using the following command:

```
ps aux | head
```

The relevant part of the output was:

```
USER  PID %CPU %MEM    VSZ   RSS TTY STAT START TIME COMMAND
root    1  0.0  0.1 21868 12504 ?   Ss   15:34 0:00 /sbin/init
```

Therefore, the PID of process number 1 is **1**, and on this WSL system it runs `/sbin/init`. PID 1 is the first process started in the Linux environment and acts as the initial process of the system; in this environment it is associated with the `systemd` initialization system.

3.2.2 Number of processes running on the idle system

The approximate number of processes was checked with the following command, which returned the value 55:

```
ps aux | wc -l
```

The `top` command reported the following summary:

```
Tasks:  53 total,   1 running,  52 sleeping,   0 stopped,   0 zombie
```

Therefore, approximately **53 processes** were running according to `top`. The small difference from the value reported by `ps aux | wc -l` is caused by the header line included in the `ps` output and by the different moments at which the two commands take their snapshot. Most processes were sleeping, which is expected for an idle system.

3.2.3 State of the sleeping process

A background `sleep` process was created and its PID was stored in a variable:

```
sleep 300 &
PID=$!
```

Its status information was inspected in the `/proc` filesystem:

```
cat /proc/$PID/status | head -20
```

The relevant line was `State: S (sleeping)`. The state `S` indicates that the process is waiting for an event or condition (in this case the expiry of a timer) rather than actively using the CPU. The process was terminated afterwards using `kill $PID`. The inspection of processes and system activity is shown in figure 3.3.

```
adi@Pikachu:~/os-lab1/demo$ top
top - 16:45:40 up  1:11,  1 user,  load average: 0.06, 0.02, 0.00
Tasks:  53 total,   1 running,  52 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7790.2 total,   6757.4 free,    742.4 used,    487.0 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7047.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    757 adi       20   0   50388  10940   7680 S   0.3   0.1   0:21.74 ijent
      1 root      20   0   21868  12504   9304 S   0.0   0.2   0:00.96 systemd
      2 root      20   0    3120   1920   1920 S   0.0   0.0   0:00.01 init-systemd(Ub
      7 root      20   0    3176   2056   1792 S   0.0   0.0   0:01.08 init
     53 root      19  -1   50456  15360  14592 S   0.0   0.2   0:00.31 systemd-journal
    106 root      20   0   25608   6400   4864 S   0.0   0.1   0:00.87 systemd-udevd
    117 root      20   0  152940   1284   1152 S   0.0   0.0   0:00.00 snapfuse
    118 root      20   0  152940   1412   1280 S   0.0   0.0   0:00.00 snapfuse
    120 root      20   0  153072   1540   1408 S   0.0   0.0   0:00.00 snapfuse
    131 root      20   0  153072   1412   1280 S   0.0   0.0   0:00.00 snapfuse
    140 root      20   0  153072   1796   1408 S   0.0   0.0   0:00.00 snapfuse
    141 root      20   0  152940   1280   1280 S   0.0   0.0   0:00.00 snapfuse
    147 root      20   0  152940   1284   1152 S   0.0   0.0   0:00.00 snapfuse
    153 root      20   0  153072   1412   1280 S   0.0   0.0   0:00.03 snapfuse
    229 systemd+  20   0   21476  12544  10624 S   0.0   0.2   0:00.40 systemd-resolve
    230 systemd+  20   0   91036   7680   6784 S   0.0   0.1   0:00.42 systemd-timesyn
    243 root      20   0    4240   2688   2432 S   0.0   0.0   0:00.02 cron
    244 message+  20   0    9664   4864   4480 S   0.0   0.1   0:00.12 dbus-daemon
    250 root      20   0 2220460  37876  24448 S   0.0   0.5   0:01.16 snapd
    251 root      20   0   17968   8448   7552 S   0.0   0.1   0:00.09 systemd-logind
    266 syslog    20   0  222516   5248   4352 S   0.0   0.1   0:00.08 rsyslogd
```

Figure 3.3 – Process inspection and system activity

The summary produced by `top` demonstrates that an apparently idle Linux system still contains many processes, most of which are waiting in a sleeping state.

3.3 Memory and swap inspection

3.3.1 Total and free RAM

The total and available memory of the system were checked using `free -h`. The output was:

```
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       736Mi       6.6Gi        21Mi       487Mi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
```

The main memory values are summarized in table 3.2.

Table 3.2 – Memory values of the system

Parameter	free -h	/proc/meminfo
Total RAM	7.6 GiB	7977176 kB
Free RAM	6.6 GiB	6922876 kB
Available RAM	6.9 GiB	7220980 kB
Buffers	–	31880 kB
Cached	–	388592 kB

Parameter	free -h	/proc/meminfo
Swap cached	–	0 kB

The values in the third column were obtained with the command `cat /proc/meminfo | head -6`, and they confirm the output of `free -h`.

3.3.2 Swap and its configuration

Swap is a portion of storage space that the operating system uses as additional virtual memory when RAM is insufficient, allowing inactive memory pages to be moved out of RAM temporarily. On the examined system the configured swap is **2.0 GiB**, of which **0 B** is used and **2.0 GiB** is free. Since plenty of RAM was available, no swap space was in use at the time of measurement.

3.3.3 Resident memory of a sleep process

A simple `sleep` process was started in the background and its resident memory was checked:

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
```

The result was `VmRSS: 1792 kB`. Therefore, a bare `sleep` process uses approximately **1792 kB (1.75 MiB)** of resident memory. This is somewhat surprising, because the command performs almost no visible work; nevertheless, even a simple process requires memory for its executable code, shared libraries, stack and kernel metadata. The process was terminated afterwards using `kill $PID`. The memory and swap information is shown in figure 3.4, and the file information observed during the work is shown in figure 3.5.

```
adi@Pikachu:~/os-lab1/demo$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       736Mi       6.6Gi        21Mi       487Mi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
adi@Pikachu:~/os-lab1/demo$ cat /proc/meminfo | head -6
MemTotal:        7977176 kB
MemFree:         6922876 kB
MemAvailable:    7220980 kB
Buffers:           31880 kB
Cached:           388592 kB
SwapCached:            0 kB
adi@Pikachu:~/os-lab1/demo$ sleep 300 & PID=$!
[3] 21450
adi@Pikachu:~/os-lab1/demo$ grep VmRSS /proc/$PID/status
VmRSS:      1792 kB
adi@Pikachu:~/os-lab1/demo$ kill $PID
adi@Pikachu:~/os-lab1/demo$
```

Figure 3.4 – Memory and swap information displayed by the system

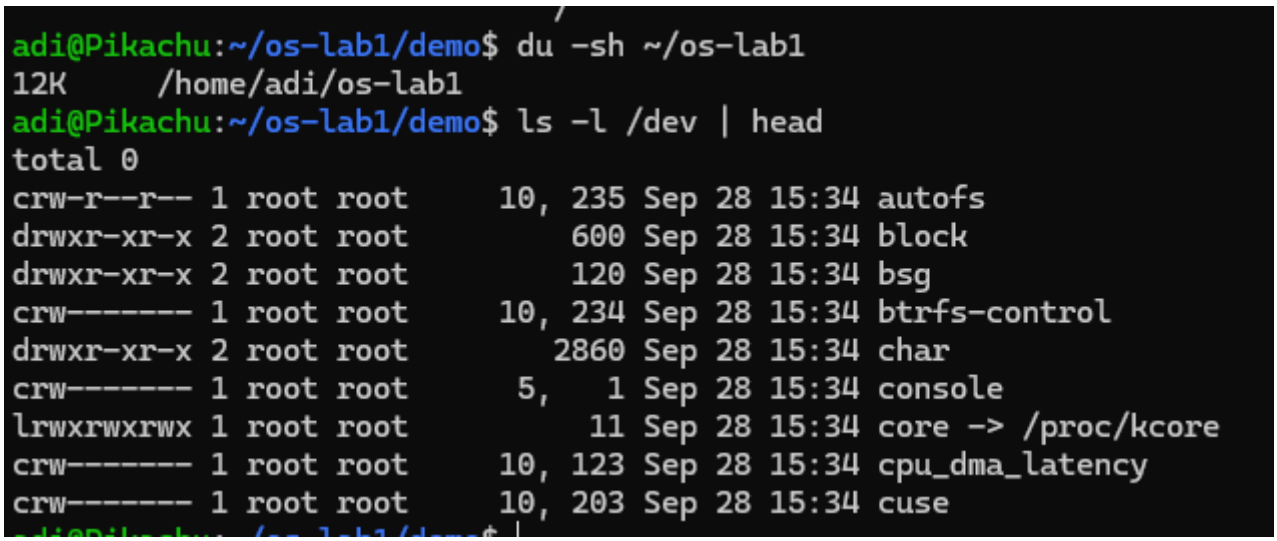


Figure 3.5 – File information and storage contents observed during the work

3.4 Linux filesystems and device files

3.4.1 Root filesystem device

The mounted filesystems were inspected using `df -h`. The relevant output was:

```
/dev/sdd        1007G   45G  911G   5% /
```

This shows that the root filesystem `/` is mounted on `/dev/sdd`. The block devices were also listed using `lsblk`:

```
sdd   8:48   0     1T  0 disk /
```

Therefore, the root filesystem is located on the **1 TB** block device `/dev/sdd`, which demonstrates the relationship between a physical block device and the root of the directory tree.

3.4.2 Example of an entry from /dev

The contents of the `/dev` directory were inspected using `ls -l /dev | head`. One of the entries found was:

```
crw------- 1 root root 5, 1 Sep 28 15:34 console
```

The `/dev/console` device represents the system console and provides an interface for system-level input and output. The letter `c` at the beginning of the line indicates a character device, meaning that data is handled as a stream of characters rather than as blocks. The filesystem and device information is shown in figure 3.6.

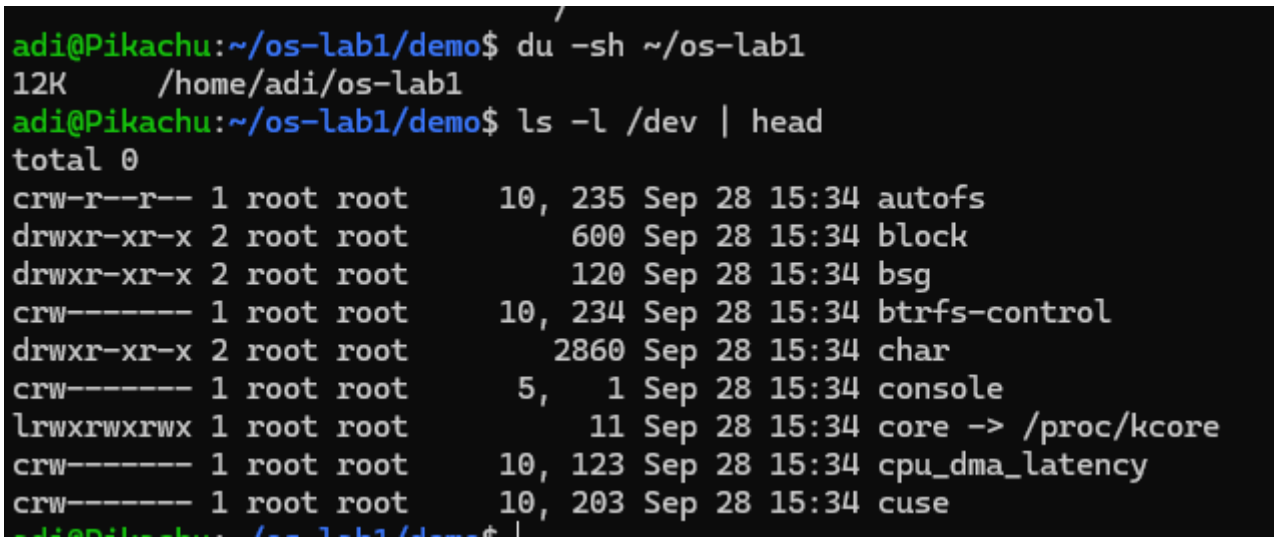


Figure 3.6 – Filesystem and device information

3.4.3 Meaning of "everything is a file"

The principle "everything is a file" means that Linux provides file-like interfaces not only for ordinary files and directories, but also for devices and other system resources, so that all of them can be accessed with the same standard file operations.

4 EXPLORING THE XV6 OPERATING SYSTEM

4.1 The xv6 shell and user programs

4.1.1 Programs provided by xv6

The xv6 environment was inspected using the `ls` command, which displayed the available user programs, as shown in figure 4.1. Three examples are:

- `cat` displays the contents of files;
- `echo` prints text to the console;
- `grep` searches for matching text in its input.

The output of `ls` also lists other programs such as `forktest`, `kill`, `mkdir`, `rm`, `sh`, `wc` and `usertests`.

```
adi@Pikachu:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

Figure 4.1 – xv6 shell and available user programs

4.1.2 Operating system features required by a pipe

For a pipe (`|`) to work inside xv6, the following two operating system features must exist:

a) processes, because the shell creates and manages a separate process for the command on each side of the pipe;
b) inter-process communication, because the pipe is the mechanism through which one process sends its output directly to another process as input.

For example, in the command `ls | grep c` the `ls` process produces the output, while the `grep` process reads that output through the pipe.

4.1.3 Comparison with the Linux shell

The xv6 shell provides the same basic command-line functionality as the Linux shell used in the first laboratory work, including running programs and connecting them with pipes, but it is much simpler and offers far fewer features.

4.2 System calls and the xv6 source code

4.2.1 System calls used by user/cat.c

The system calls involved in the execution of `user/cat.c` and the service each of them requests from the kernel are listed in table 4.1.

Table 4.1 – System calls involved in the execution of cat

System call	Service requested from the kernel
`read()`	Reads data from a file descriptor into a buffer
`write()`	Writes data from a buffer to a file descriptor, such as the console
`open()`	Opens a file and returns a file descriptor
`close()`	Closes an open file descriptor
`exit()`	Terminates the current process
`fork()`	Creates a new process when the shell executes a command
`exec()`	Replaces the current process image with the specified user program

Inside the `cat()` function itself, the main operations are `read()` and `write()`: `read()` obtains data from a file descriptor, while `write()` outputs that data to the console. The declarations of all available system calls can be found in the header file `user/user.h`, as shown in figure 4.2.

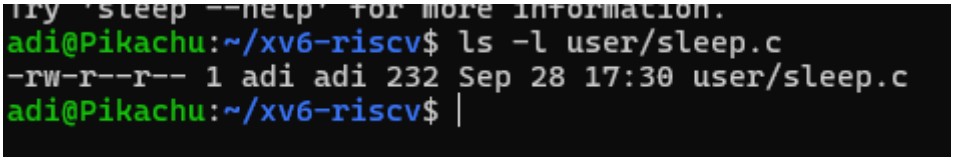


Figure 4.2 – System call declarations in user/user.h

4.2.2 Location of sys_read in the kernel

The `grep` command was used to locate the implementation of the system call in the file `kernel/sysfile.c`. The result was:

```
69:sys_read(void)
83:sys_write(void)
```

Therefore, `sys_read` is implemented in the file **kernel/sysfile.c** on **line 69**.

4.2.3 Difference between kernel/ and user/

The code in `kernel/` implements the operating system and its low-level services, while the code in `user/` contains programs that run in user space and request kernel services through system calls.

4.3 Creating the sleep user program

A new user program named `sleep` was added to xv6. The program accepts a number of clock ticks as its argument and calls the existing `sleep()` system call to pause execution. The source file was created at `~/xv6-riscv/user/sleep.c` and contains the following code:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

The program first checks that exactly one argument was supplied and prints a usage message to the standard error stream otherwise. The argument is converted to an integer with `atoi()` and passed to the `sleep()` system call, after which the process terminates with `exit(0)`.

After the file was created and xv6 was rebuilt, the presence of the file in the `user/` directory was verified, as shown in figure 4.3. The file submitted for this work is `sleep.c`, located at `/home/adi/xv6-riscv/user/sleep.c`, and it can be checked with the command `ls -l ~/xv6-riscv/user/sleep.c`.

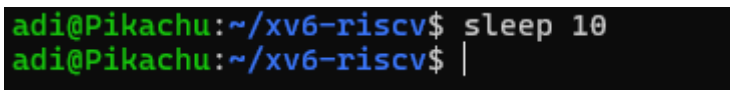


Figure 4.3 – The user/sleep.c file created in the xv6 source tree

4.4 Testing sleep inside xv6

Figure 4.3 was taken in the WSL terminal, outside xv6, so it does not show the new program running inside xv6. Because that WSL setup is no longer available, the test was repeated on my current laptop (Fedora 44) with the same source tree `~/xv6-riscv`. In the meantime (August 2025) the xv6 authors renamed the system call `sleep()` to `pause()`, so the current `user/sleep.c` calls `pause()`; everything else in the program is unchanged (the change is described in Lab 4).

`sleep 10` paused the xv6 shell for 10 ticks and then the prompt came back, and `sleep` without an argument printed the usage message.

```
adelina@192:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel (was sleep() before Aug 2025)
  exit(0);
}
adelina@192:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nogra
ve file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mm

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
$ QEMU: Terminated
adelina@192:~/xv6-riscv$
```

Figure 4.4 – sleep running inside xv6, with and without an argument

CONCLUSION

In the course of the two practical works, the four main resources managed by an operating system, namely files, processes, memory and devices, were examined first on a full Linux system and then on the xv6 teaching kernel.

In the first work, file ownership and permissions were observed with `ls -l`, and the mode `600` was shown to restrict access exclusively to the owner. Processes were inspected with `ps aux`, `top` and the `/proc` filesystem, which revealed that an idle system still runs about 53 processes, most of them in the sleeping state. Memory was analyzed with `free -h` and `/proc/meminfo`, showing 7.6 GiB of RAM with 2.0 GiB of unused swap, and even a trivial `sleep` process was found to occupy about 1.75 MiB of resident memory. Devices were observed with `lsblk` and `ls -l /dev`, which illustrated the principle that "everything is a file".

In the second work, the xv6 shell and its user programs were explored, and the use of pipes demonstrated the role of processes and inter-process communication. The analysis of `user/cat.c`, `user/user.h` and `kernel/sysfile.c` showed how user programs request kernel services through system calls, with `sys_read` implemented on line 69 of `kernel/sysfile.c`. Finally, a new `sleep` user program was implemented, which confirmed that a user-space program can make use of existing kernel functionality solely through the system call interface.

REFERENCES

1. chmod(1) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man1/chmod.1.html
2. Filesystem Hierarchy Standard, version 3.0. The Linux Foundation. [online]. Available: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
3. proc(5) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html
4. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system. MIT. [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
5. mit-pdos/xv6-riscv – source code repository. [online]. Available: https://github.com/mit-pdos/xv6-riscv